30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 17

Performance — Part 1: Rendering, Jank, Memory & Profiling



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 17

Why performance is Day 17

Users never see your architecture — they feel dropped frames, a heavy heap and a hot phone. In 2026 performance questions test one habit: **measure before you guess**.

- Frame deadlines follow the refresh rate — 60, 90 or 120 Hz
- Compose performance is about **which phase reads state**, not "fewer composables"
- Leak, churn and bloat are three memory problems with three different fixes
- Perfetto is the system tracer; Systrace is legacy
- Profile a **release-like** build and verify the fix with the same metric

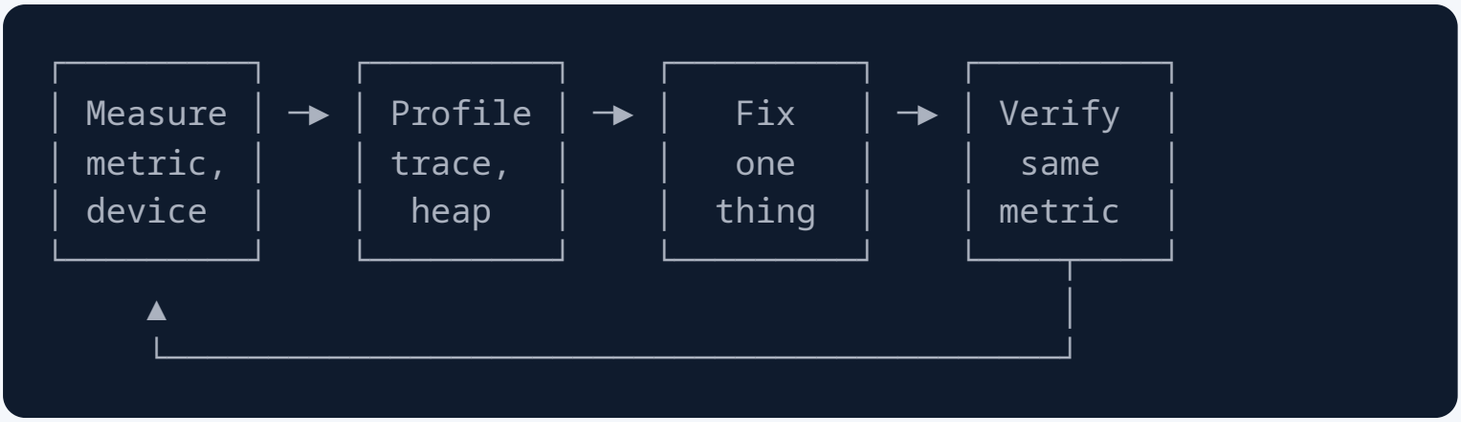
Level	Questions
Junior	Jank & the frame pipeline · Memory basics · Overdraw & layout cost
Mid	Compose phases · Derived state & lists · LeakCanary · Studio profilers
Senior	Perfetto jank analysis · Heap & native memory · Compose toolkit
Architect	Main-thread budget · Memory-aware caching · Lab + field profiling



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 17

Mental model: measure → profile → fix → verify



- **Measure** a user-visible number first: janky frames, P90 frame time, heap after a journey
- **Profile** to find the cause — the metric says *that* it's slow, the trace says *where*
- **Fix one thing**, then re-measure on the same device with the same build type

RULE OF THUMB

A performance claim without a number, a device and a build type is an opinion. "It feels faster" is not a result.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 17

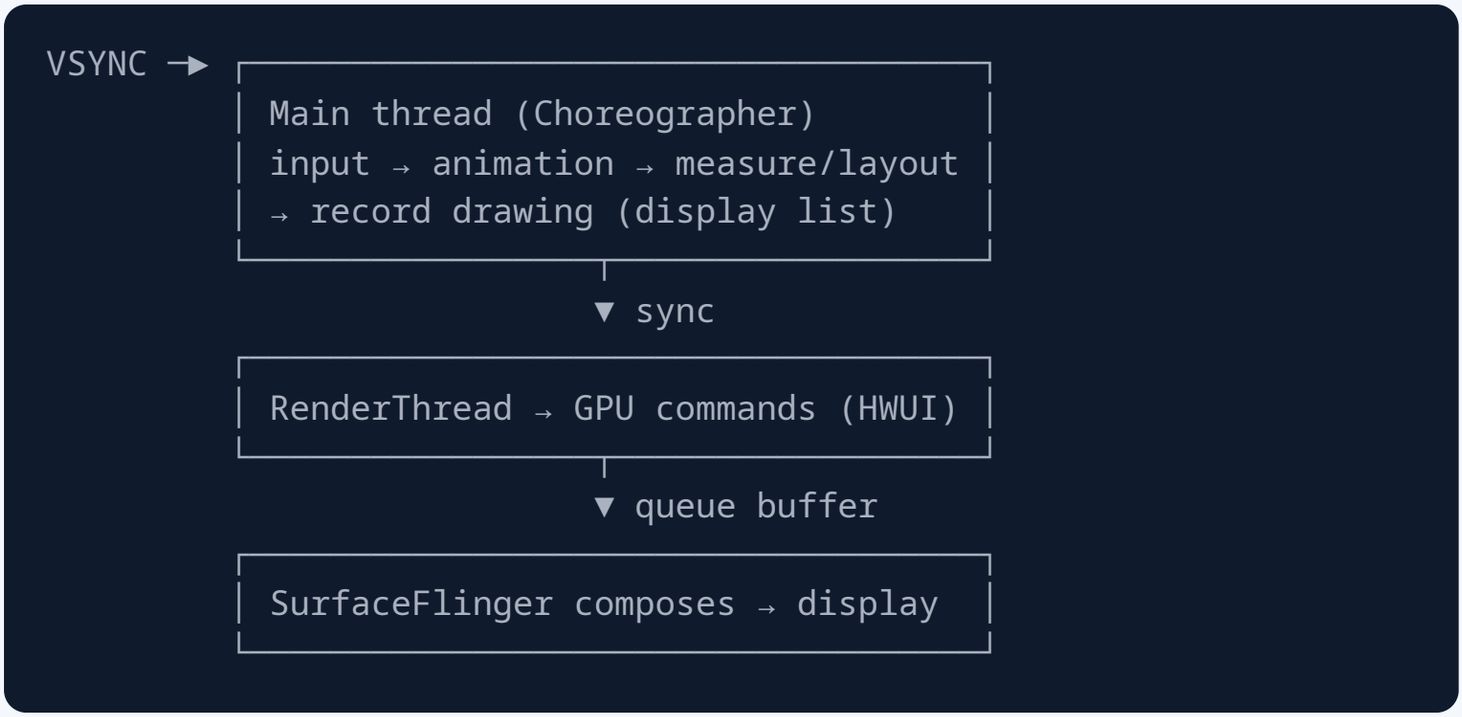
Junior

Jank & the frame pipeline · Memory basics · Overdraw & layout cost



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What is jank? The frame pipeline



- The **main thread** runs input, animations and layout, then only *records* draw commands
- The **RenderThread** turns them into GPU work; **SurfaceFlinger** composes every layer on screen
- **Jank** = a frame that misses its deadline, so the previous frame stays on screen



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. The budget depends on the display

Refresh rate	Frame deadline
60 Hz	≈ 16.7 ms
90 Hz	≈ 11.1 ms
120 Hz	≈ 8.3 ms

- Both the main-thread work and the RenderThread work for a frame must fit, or the frame is late
- A **frozen frame** — longer than 700 ms in Android vitals — feels like a hang, not a stutter

INTERVIEW TRAP

"The frame budget is 16 ms." — Only at 60 Hz. On a 120 Hz phone the same code has half the time. Think in frames, and test at the device's real refresh rate.

INTERVIEW TRAP

"The main thread draws the pixels." — It records drawing commands; RenderThread and the GPU rasterize them. A heavy `onDraw` or `drawBehind` costs main-thread time; big layers and heavy effects cost RenderThread/GPU time.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Memory basics: leak, churn, bloat

Problem	What you see	Typical fix
Leak	Heap grows each time you repeat a journey; old screens stay alive	Fix the reference that outlives its owner (Day 1)
Churn	Saw-tooth graph, frequent GCs during scroll or animation	Stop allocating in hot paths: `onDraw`, `onBindViewHolder`, composition
Bloat	Heap is high from the start	Size caches, downsample bitmaps, drop data you don't show

- The Java/Kotlin heap has a per-app limit (`Runtime.getRuntime().maxMemory()`); exceeding it throws `OutOfMemoryError`
- Native memory isn't capped by that limit, but it's still part of the process footprint the system watches under memory pressure



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Two myths from last year's answers

INTERVIEW TRAP

"Since Android 8, bitmaps live on the Java heap." — Reversed. From Android 3.0 to 7.1 pixel data lived on the Java heap; **since Android 8.0 (API 26) it lives in the native heap**. A bitmap-heavy app can look fine in the Java heap graph and still use a lot of memory.

INTERVIEW TRAP

"GC freezes the main thread, like the JVM's G1 or CMS." — ART isn't HotSpot. Since Android 8 it uses a **concurrent copying** collector with very short pauses. Churn still hurts: GC work burns CPU and battery and competes with your frames.

FOLLOW-UP

"Why is my app killed in the background without any OOM?" — That's the low-memory killer reclaiming cached processes (Day 1). A smaller footprint helps, but you must still restore state correctly.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Overdraw: paint each pixel once

Developer options → **Debug GPU overdraw**:

Color	Meaning
True color	Drawn once
Blue	Overdrawn once
Green	Overdrawn twice
Pink	Overdrawn three times
Red	Overdrawn four or more times

- Fix: remove backgrounds nobody sees (window, root and card all painting the same color); don't stack opaque full-screen layers
- **Profile HWUI rendering** draws per-frame bars against the deadline — a quick sniff test, not an analysis
- Both work for Views **and** Compose: they share the same HWUI rendering pipeline



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Layout cost: Views vs Compose

- **Views:** the cost is repeated measure passes. `RelativeLayout` and weighted `LinearLayout` can measure children twice; nesting them multiplies it.
- **Compose:** each child is measured **once per layout pass** — measuring twice throws. Intrinsic measurements are the explicit exception. Deep nesting is cheap.

INTERVIEW TRAP

"ConstraintLayout is always faster — flatten everything." — It removes double measuring, but its solver isn't free. A shallow `LinearLayout` without weights can be cheaper. Decide with a trace, not by habit.

INTERVIEW TRAP

"Overdraw is the main cause of jank." — On modern GPUs it rarely is. Look at main-thread work first: inflation, binding, allocation, disk I/O, lock contention.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 17



Mid-Level

Compose phases · Derived state & lists · LeakCanary · Studio profilers



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Compose: read state as late as possible

Phase	Answers	A state read here re-runs
Composition	*What* to show	composition → layout → drawing
Layout	*Where*: measure and place	layout → drawing
Drawing	*How* it looks	drawing only

- The lambda versions of `offset`, `graphicsLayer`, `drawBehind` and `layout` read state in a later phase
- Values that change every frame (scroll, animation) should never be read in composition if they only move, fade or recolor something

INTERVIEW TRAP

"Fewer composables means faster UI." — Composables are cheap. What's expensive is re-running a large scope on every frame. Split scopes and defer reads.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Deferring reads in code

```
// BAD: the animated value is read in composition,
// so the Box recomposes on every animation frame
val offsetY by animateDpAsState(target)
Box(Modifier.offset(y = offsetY))

// GOOD: read in the layout phase, no recomposition
val offsetPx by animateFloatAsState(targetPx)
Box(Modifier.offset { IntOffset(0, offsetPx.roundToInt()) })

// GOOD: read in the draw phase, no layout either
val color by animateColorAsState(targetColor)
Box(Modifier.fillMaxSize().drawBehind { drawRect(color) })
```

With `by`, the `State` is read where the variable is used — inside the lambda, so in a later phase.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Derived state and lazy lists

```
// Recomposes only when the Boolean flips
val showToTop by remember {
    derivedStateOf { listState.firstVisibleItemIndex > 0 }
}

LazyColumn(state = listState) {
    items(
        items = posts,
        key = { it.id },           // stable identity
        contentType = { it.kind }, // reuse per type
    ) { post -> PostRow(post) }
}
```

- `key`: inserts and moves keep item state and don't recompose every row
- `contentType`: item compositions are reused only between items of the same type
- Sorting and filtering belong in the ViewModel, not in the composable body



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Three classic Compose mistakes

INTERVIEW TRAP

`derivedStateOf` everywhere. It pays off only when the output changes **less often** than its inputs. `derivedStateOf { first + last }` just adds overhead.

INTERVIEW TRAP

Backwards writes: writing a state you already read in the same composition (e.g. `count++` in the body) invalidates it again — an endless recomposition loop. Write state in event handlers and effects.

INTERVIEW TRAP

Judging Compose performance in a **debug build**. Debuggable apps skip ART optimizations and add debug overhead. Measure a release build with R8 and a Baseline Profile (Day 18).

FOLLOW-UP

"Do I still need `@Stable` everywhere?" — No. Strong skipping is on by default since Kotlin 2.0.20 (Day 8). Fix stability only where counts or traces show wasted work.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. LeakCanary: how it finds leaks

```
dependencies {
    debugImplementation(libs.leakcanary.android)
    // optional: fail instrumented tests on leaks
    androidTestImplementation(libs.leakcanary.instrumentation)
}
```

- No code needed. It watches objects that should die: destroyed Activities and Fragments, Fragment views after `onDestroyView`, cleared ViewModels, destroyed Services
- If an object is still reachable 5 s later and after a GC, it's **retained**. When enough pile up, LeakCanary dumps the heap and **Shark** finds the shortest strong reference path from a GC root
- Watch your own objects: `AppWatcher.objectWatcher.expectWeaklyReachable(obj, "reason")`



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Reading a leak trace

```
┬───
│ GC Root: System class
│
├─ com.example.SessionManager class
│    Leaking: NO (a class is never leaking)
│    ↓ static SessionManager.listener
│                            ~~~~~~~~
├─ com.example.FeedFragment$onViewCreated$1 instance
│    Leaking: UNKNOWN
│    ↓ FeedFragment$onViewCreated$1.this$0
│                                   ~~~~~~
╰→ com.example.FeedFragment instance
     Leaking: YES (Fragment#onDestroy() was called)
```

- Read from the GC root down. The bad reference is between the last `Leaking: NO` and the first `Leaking: YES`; suspects are underlined with `~~~`
- Here a singleton kept a listener that captured the Fragment (`this$0`). Fix: unregister in `onDestroyView`, or expose a `Flow` and collect it with `viewLifecycleOwner`

INTERVIEW TRAP

"LeakCanary finds native memory leaks too." — It analyzes the Java/Kotlin heap only. For native growth, record native allocations in Android Studio or Perfetto (heapprofd).



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Android Studio profilers: pick the task by symptom

Symptom	Start with
Janky scroll or animation	System trace (Perfetto)
High CPU, a hot method	CPU callstack sampling
Memory keeps growing	LeakCanary, then a heap dump
Frequent GCs	Java/Kotlin allocation tracking
Native memory growth	Native allocation recording
Recomposition storms	Layout Inspector recomposition counts

- For CPU and timing, profile a **profileable release build** ("Profile with low overhead"). Debuggable builds disable many ART optimizations — their timings lie
- Hunting leaks in a debug build is fine: a leaked reference doesn't depend on compiler optimizations



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Sampling, tracing and your own sections

```
<!-- AndroidManifest.xml of the release-like build -->
<application>
    <profileable android:shell="true" />
</application>
```

```
import androidx.tracing.trace

suspend fun loadFeed(): Feed {
    val body = api.fetchFeed()
    return trace("Feed.parse") {
        json.decodeFromString<Feed>(body)
    }
}
```

- Your sections appear in Perfetto and Studio traces; Macrobenchmark can measure them (Day 18)
- **Method tracing** instruments every call: counts are right, timings inflated. **Sampling** gives realistic timings but can miss short calls

INTERVIEW TRAP

"Custom trace sections need Android 10+." — `android.os.Trace.beginSection` exists since API 18. `androidx.tracing` adds the Kotlin `trace {}` helper.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 17



Senior

Perfetto jank analysis · Heap & native memory · Compose toolkit



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Perfetto: record a jank trace

Record from Android Studio (*system trace*), the on-device **System Tracing** app (Developer options, Quick Settings tile), or `adb`:

```
adb shell perfetto -o /data/misc/perfetto-traces/jank \
  -t 10s sched freq gfx view input am wm dalvik
adb pull /data/misc/perfetto-traces/jank
# open the file in ui.perfetto.dev
```

Android 12+ adds **FrameTimeline**: an *Expected* and an *Actual* timeline per app, with a jank type on every late frame.

Systrace-era habit	2026
`systrace.py`	Perfetto: Studio, System Tracing app, `perfetto` CLI
"Green line = 16 ms"	The deadline follows the refresh rate
`dumpsys gfxinfo` janky %	A quick check; FrameTimeline tells you *why*



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Read it: from the late frame to the cause

- Start at a late frame in **Actual Timeline**. *App Deadline Missed* → your threads. *SurfaceFlinger...* or *Buffer Stuffing* → not only your code
- Then check the main thread's **state** during that frame:

Main thread state	Usually means
Running, long `doFrame`	Too much work: inflation, binding, recomposition
Runnable	Waiting for a CPU — contention, too many busy threads
Sleeping on a lock or binder	Waiting for another thread or process
Uninterruptible sleep	Blocking I/O — disk on the main thread

```
SELECT name, dur / 1000000 AS ms
FROM slice
WHERE name LIKE 'Choreographer#doFrame%'
ORDER BY dur DESC
LIMIT 5;
```

INTERVIEW TRAP

"The long `doFrame` is the cause." — It's the symptom. Expand its children (`inflate`, `RV OnBindView`, `Recomposer:recompose`, `Lock contention on ...`) and see what the thread was waiting for.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Heap dumps: shallow vs retained

- Capture with Studio's heap dump task, or `adb shell am dumpheap <package> /data/local/tmp/app.hprof`
- **Shallow size** = the object itself. **Retained size** = what the GC would free if this object died. Sort by retained size.
- Filter for Activity/Fragment leaks, then follow **references** up to the GC root
- `adb shell dumpsys meminfo <package>` splits the footprint: Java heap, native heap, graphics, code — it tells you *which* memory grew

INTERVIEW TRAP

"The Java heap looks fine, so memory is fine." — The Java heap is one slice. Bitmap pixels (API 26+), native libraries and graphics buffers live outside it. Check `dumpsys meminfo` and PSS.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Bitmaps and native memory

```
Photo  4000 × 3000 px × 4 B (ARGB_8888)
       = 48,000,000 B  ≈ 46 MiB
Thumb  400 × 300 dp on xxhdpi (×3) = 1200 × 900 px
       = 4,320,000 B   ≈ 4 MiB
```

- Decode at display size: Coil and Glide do it for you; with `BitmapFactory`, use `inSampleSize`
- `Bitmap.Config.HARDWARE` keeps pixels only in graphics memory — cheaper, but immutable and not readable by the CPU
- Native growth (bitmaps, JNI, codecs): Studio's **native allocation** recording uses heapprofd (Android 10+)

FOLLOW-UP

"How do you prove a leak is fixed?" — Repeat the journey several times, force a GC, dump the heap: the count of that Activity must go back to what's on screen. Then keep a LeakCanary instrumented test so it stays fixed.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. The Compose performance toolkit

Tool	Answers
Layout Inspector recomposition counts	Which composables recompose or skip, and how often
Compose compiler reports	Which parameters are unstable, which functions are skippable
Composition tracing (`runtime-tracing`)	Composable names inside system traces
Macrobenchmark `FrameTimingMetric`	Did the fix change frame times? (Day 18)

```
// build.gradle.kts, Compose Compiler Gradle plugin
composeCompiler {
    reportsDestination = layout.buildDirectory.dir("compose")
    metricsDestination = layout.buildDirectory.dir("compose")
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Using the numbers well

- Types from modules without the Compose compiler count as unstable; a **stability configuration file** can declare them stable — a promise that they don't mutate
- Pass fast-changing values as lambdas (`() -> Float`) so only the reader recomposes

INTERVIEW TRAP

"High recomposition counts must be fixed." — Only if frames suffer. Counts guide the search; frame times decide.

INTERVIEW TRAP

"Mark everything `@Immutable` to be safe." — That's lying to the compiler. If the object does change, the UI skips the update and shows stale data.

FOLLOW-UP

"Did strong skipping make stability irrelevant?" — No. Unstable parameters are compared by instance (`===`), so a new-but-equal list on each emission still recomposes.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 17

Architect

Main-thread budget · Memory-aware caching · Lab + field profiling



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. The main-thread budget as architecture

The main thread gets input, state-to-UI mapping, layout and draw recording. Everything else has a home:

Work	Home
Disk, database, network	Room and Ktor/Retrofit `suspend` APIs — already main-safe
Parsing, sorting, diffing	`Dispatchers.Default`
Image decoding	The image library's threads (Coil, Glide)
SDK initialization	Lazily, on first use — not in `Application.onCreate`
Key-value storage	DataStore instead of `SharedPreferences`

- StrictMode in debug builds catches disk and network on the main thread (Day 1)
- A shared tracing convention (`trace("Feature.step")`) makes every team's work visible in the same Perfetto trace



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. "Just move it off the main thread" isn't enough

INTERVIEW TRAP

"Move work to a background thread and the jank is gone." — Background work still hurts frames when it competes for the few fast CPU cores, holds a lock the main thread needs (*Lock contention* in the trace), posts results in bursts that invalidate the whole screen, or allocates enough to keep the GC busy.

INTERVIEW TRAP

"App Startup runs initializers in parallel on a background thread." — It runs them **on the main thread, one after another**, in dependency order. Its win is replacing many ContentProviders with one.

INTERVIEW TRAP

"`SharedPreferences.apply()` *is async, so it's harmless."* — Android's own docs note that `apply()` can block the UI thread on `fsync()`. Prefer DataStore.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Memory-aware caching

```
class ThumbnailCache(maxBytes: Int) :
    LruCache<String, Bitmap>(maxBytes), ComponentCallbacks2 {

    override fun sizeOf(key: String, value: Bitmap): Int =
        value.allocationByteCount

    override fun onTrimMemory(level: Int) {
        when {
            level >= TRIM_MEMORY_BACKGROUND -> evictAll()
            level >= TRIM_MEMORY_UI_HIDDEN ->
                trimToSize(maxSize() / 2)
        }
    }
    // onConfigurationChanged / onLowMemory: omitted
}
// appContext.registerComponentCallbacks(cache)
```

- Size caches in **bytes**, not entries; derive the budget from `ActivityManager.memoryClass`
- Android 14 (API 34) deprecated the `TRIM_MEMORY_RUNNING_*` levels — apps no longer receive them. React to `UI_HIDDEN` and `BACKGROUND`



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Lifetimes are a memory decision

- Screen data in ViewModels; app data in repositories backed by disk — not in singletons
- **One** image pipeline for the app (one Coil `ImageLoader`), with one memory budget
- Large lists: hold a window with Paging 3 (Day 6), never the whole dataset

INTERVIEW TRAP

"*`android:largeHeap="true"` fixes OOMs.*" — It raises the heap limit on some devices, hides the leak, and makes GC and background kills costlier. Fix the footprint.

INTERVIEW TRAP

"Make it `@Singleton` so it's fast." — A singleton lives as long as the process: every cache and listener inside it becomes permanent memory.

INTERVIEW TRAP

"Object pools everywhere." — ART allocates short-lived objects cheaply. Pool only where a profiler shows churn in a hot path such as `onDraw`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. A profiling method that survives interviews

1. **Reproduce** on a real mid- or low-end device with a release-like build (R8, profileable)
2. **Measure** the journey — Macrobenchmark or FrameTimeline — and write the baseline number down
3. **Trace** to find the cause; test one hypothesis at a time
4. **Fix**, re-measure the same metric, keep the benchmark as a regression test (Day 18)
5. **Confirm in the field**: JankStats, Android vitals, `ProfilingManager` traces (Android 15+)

INTERVIEW TRAP

"Lab numbers are what users feel." — The lab is repeatable; the field is the truth. Device mix, thermal state, background load and data sizes differ. You need both.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Field jank with context: JankStats

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent { FeedScreen() }

    PerformanceMetricsState
        .getHolderForHierarchy(window.decorView)
        .state?.putState("screen", "feed")

    jankStats = JankStats.createAndTrack(window) { frame ->
        if (frame.isJank) reporter.record(
            durationNs = frame.frameDurationUiNanos,
            states = frame.states.map { it.key to it.value },
        )
    }
}
// onResume/onPause: jankStats.isTrackingEnabled = true/false
```

- The listener runs for every frame and its `FrameData` is reused — copy what you keep, sample and batch before upload
- UI state tags ("screen", "scrolling") make a janky frame attributable to a screen

SENIOR ANSWER IN ONE LINE

"Measure a user-visible metric on a real device with a release build, trace to find the cause, fix one thing, and verify with the same metric — in the lab and in the field."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 17 · ARCHITECT

Day 17 — cheat sheet

- Frame deadline = 1 / refresh rate: 16.7, 11.1 and 8.3 ms at 60, 90 and 120 Hz.
- Main thread records drawing; RenderThread and GPU rasterize; SurfaceFlinger composes.
- Leak ≠ churn ≠ bloat. Bitmap pixels live in the **native** heap since API 26.
- ART's GC is concurrent — churn costs CPU and battery, not long pauses.
- Compose: read fast-changing state in the latest phase (`offset {}`, `graphicsLayer {}`, `drawBehind {}`).
- `derivedStateOf` only when the output changes less often; `key` + `contentType` in lazy lists.
- LeakCanary = Java heap leaks, in debug and in CI. Native memory → heapprofd.
- Profile profileable release builds; sampling for timings, tracing for call counts.
- Perfetto + FrameTimeline replaced Systrace: start at the late frame, then the thread state.
- Measure → profile → fix one thing → verify with the same metric.

Day 18 is next → Performance, part 2: app size, battery, Baseline Profiles & Macrobenchmark. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700